

GPUMODE Lecture Study Notes: Lecture 29

Triton Internals

Core Problem the Lecture Addresses

This lecture explains what happens underneath the Python-facing surface of Triton. At the user level, Triton feels like a compact Python-style language for writing GPU kernels. A researcher writes a function, decorates it with `@triton.jit`, and calls it from PyTorch. Under the hood, Triton is a compiler stack that turns this high-level program into target-specific executable GPU code.

The central question is:

How does a Triton Python DSL kernel become executable GPU machine code?

The lecture focuses on the full path from a Triton kernel to backend-specific code:

Triton DSL $\rightarrow$ Triton IR $\rightarrow$ Triton GPU IR $\rightarrow$ LLVM IR $\rightarrow$ PTX $\rightarrow$ cubin $\rightarrow$ loaded GPU kernel.

The main systems problem is that a high-level tensor-block program must be lowered into efficient hardware-specific instructions while preserving the programmer's intent. Triton must handle pointer arithmetic, masking, memory operations, tensor layouts, warp-level execution, backend-specific code generation, runtime launch logic, and caching.

The important mental shift is that Triton is not just nicer CUDA syntax. Triton is a compiler and runtime system. The visible Python DSL is only the frontend contract; the real work happens through IR lowering, compiler passes, target-specific backends, and just-in-time compilation.

Required Background

CUDA Compilation

Traditional CUDA compilation separates CPU host code from GPU device code. Host code is compiled by a standard C or C++ compiler such as GCC or Clang. Device code is compiled through NVIDIA's CUDA compiler toolchain.

A simplified CUDA compilation path is:

$$\text{CUDA C/C++ source} \rightarrow \begin{cases} \text{host compiler path,} \\ \text{device compiler path} \end{cases} \rightarrow \text{linked executable.}$$

For device code, a useful conceptual path is:

$$\text{CUDA kernel} \rightarrow \text{PTX} \rightarrow \text{SASS} \rightarrow \text{cubin.}$$

PTX is NVIDIA's virtual GPU assembly representation. SASS is closer to the actual architecture-specific instruction stream. A cubin file is a compiled CUDA binary object that can be loaded by the CUDA driver.

PTX, SASS, and Cubin

PTX stands for Parallel Thread Execution. It is lower-level than CUDA C++, but it is not always the final code executed by the GPU.

SASS is NVIDIA's architecture-specific machine instruction representation. It depends on GPU generation and compute capability.

Cubin is a compiled CUDA binary. It is commonly stored in an ELF-like format and contains compiled device code plus metadata.

The relationship is:

$$\text{high-level kernel} \rightarrow \text{virtual GPU assembly} \rightarrow \text{architecture-specific instructions} \rightarrow \text{loadable binary.}$$

For NVIDIA-targeted Triton compilation, the practical path is:

$$\text{Triton kernel} \rightarrow \text{PTX} \rightarrow \text{cubin} \rightarrow \text{CUDA driver launch.}$$

Intermediate Representations

An intermediate representation, or IR, is a compiler-internal form of a program. IR is neither the original source code nor the final binary. It exists so the compiler can analyze, optimize, transform, and lower the program gradually.

A compiler usually applies a sequence of transformations:

$$P_0 \xrightarrow{\phi_1} P_1 \xrightarrow{\phi_2} P_2 \xrightarrow{\phi_3} \dots \xrightarrow{\phi_k} P_k.$$

Here:

- P_0 is the original program.
- Each ϕ_i is a compiler pass.
- Each P_i is a new IR-level representation.
- P_k is close to machine code or a target-specific binary format.

Correctness requires:

$$\forall x, \quad P_0(x) = P_k(x),$$

where x represents legal program inputs.

MLIR

MLIR means Multi-Level Intermediate Representation. The letters ML do not mean machine learning. MLIR is a compiler infrastructure for building compilers that need multiple abstraction levels.

MLIR provides:

- reusable compiler infrastructure;
- common optimization passes;
- custom dialect mechanisms;
- pass registration infrastructure;
- table-driven code generation through TableGen;
- tools for lowering between abstraction levels.

A useful analogy is:

$$\text{Triton DSL} : \text{MLIR} \approx \text{C/C++ source} : \text{Clang ecosystem}.$$

Triton is the programming model and compiler built using MLIR infrastructure. MLIR is the reusable compiler toolkit underneath.

Dialects

An MLIR dialect defines a set of operations, types, attributes, and semantics for a particular abstraction level. Triton uses dialects to represent Triton-specific programs and GPU-specific programs.

Important conceptual dialect layers include:

- Triton IR, which represents common Triton semantics.
- Triton GPU IR, which represents GPU-specific structure.
- Triton NVIDIA GPU IR, which contains NVIDIA-specific lowering details.
- LLVM IR, which is a lower-level compiler representation.

The point of dialects is extensibility. Triton can define custom operations while still benefiting from standard compiler infrastructure.

Main Concepts

Why Triton Matters

Triton is useful because it reduces the time between a research idea and a performant GPU implementation. Instead of writing full CUDA C++ kernels, users write compact Python-style kernels that can achieve strong performance.

The high-level workflow is:

research idea $\rightarrow$ custom tensor operation $\rightarrow$ Triton kernel $\rightarrow$ compiled GPU code $\rightarrow$ production or experiment

This is especially important for deep learning systems because new architectures often need custom kernels for attention, normalization, quantization, fused operators, sparse operations, or specialized matrix multiplication.

Triton as a Compiler Pipeline

A Triton program is lowered through multiple levels:

Triton Python DSL $\rightarrow$ Triton IR $\rightarrow$ Triton GPU IR $\rightarrow$ LLVM IR $\rightarrow$ PTX $\rightarrow$ cubin.

For NVIDIA hardware, the final result is loaded through CUDA driver APIs. For other hardware, Triton can use different backend paths.

A portable view is:

$$\text{Triton DSL} \rightarrow \text{common Triton IR} \rightarrow \begin{cases} \text{NVIDIA backend,} \\ \text{AMD backend,} \\ \text{Intel GPU backend,} \\ \text{CPU backend,} \\ \text{custom accelerator backend.} \end{cases}$$

This makes Triton interesting not only as a CUDA alternative, but also as a possible common frontend for many accelerators.

Common IR and Backend-Specific IR

The lecture emphasizes that Triton IR is intended to be a common representation. After this common layer, the lowering pipeline can diverge into hardware-specific paths.

A helpful model is:

common frontend+common optimizations+target-specific lowering+runtime launch.

Earlier versions of Triton were more NVIDIA-centered. The backend has been evolving toward a more modular architecture so that hardware-specific behavior can be separated from common Triton and GPU logic.

Triton JIT Compilation

Most users interact with Triton through just-in-time compilation. The first time a kernel is called with a particular combination of compile-time values and target properties, Triton compiles it and caches the result.

The JIT process is:

kernel call $\rightarrow$ specialization $\rightarrow$ IR generation $\rightarrow$ lowering $\rightarrow$ binary generation $\rightarrow$ cache $\rightarrow$ launch.

The compiled result can depend on:

- the source code of the kernel;
- compile-time constants;
- meta-parameters such as block size;
- tensor dtypes;
- backend target;

- GPU architecture;
- number of warps;
- dot precision settings.

Therefore, one Python kernel can produce many compiled binaries.

Triton Cache

Triton stores compilation artifacts in a cache directory. A typical default location is under a user-level Triton cache directory. The cache location can be changed using a variable such as `TRITON_CACHE_DIR`.

The cache may contain files such as:

- `.ttir` files for Triton IR;
- `.ttgir` files for Triton GPU IR;
- `.llir` files for LLVM IR;
- `.ptx` files for PTX assembly;
- `.cubin` files for compiled CUDA binaries;
- JSON metadata files;
- shared libraries for launch support;
- CUDA utility libraries.

The exact artifacts depend on whether the kernel is compiled using a command-line compilation path or JIT compilation.

Hardware-Level Explanation

Why Hardware Enters the Compiler

A Triton kernel begins as a tensor-block program, but high performance depends on GPU-specific details. The compiler must eventually decide how the program maps to hardware execution.

Important hardware concepts include:

- SMs, or streaming multiprocessors;
- warps;
- threads per warp;
- thread blocks or CTAs;
- registers;

- shared memory;
- L1 and L2 cache;
- global memory;
- memory coalescing;
- tensor cores;
- occupancy;
- scheduling and latency hiding.

The compiler lowering process progressively reveals and specializes these details.

Program IDs

In Triton, `tl.program_id` identifies the current program instance. For a one-dimensional vector addition kernel, each program instance handles a block of elements.

Let:

$$B = \text{BLOCK_SIZE}, \quad p = \text{program id.}$$

Then the offsets handled by one program instance are:

$$\text{offsets} = pB + [0, 1, 2, \dots, B - 1].$$

If the tensor has N elements, masking prevents out-of-bounds access:

$$\text{mask}_i = (pB + i) < N.$$

This is one of the most basic indexing patterns in Triton.

Warps and Threads

For NVIDIA GPUs, a warp usually contains:

$$W = 32$$

threads. If a Triton kernel is configured with n_w warps, the total thread count associated with a program instance is approximately:

$$T = n_w W.$$

The compiler may include metadata such as:

- number of warps;
- threads per warp;
- target GPU architecture;
- compute capability;
- CTA layout;
- tile shape.

These attributes matter when lowering from Triton GPU IR into lower-level backend IR.

Memory Access in Vector Addition

For vector addition:

$$Z_i = X_i + Y_i.$$

Each element requires:

- one load from X ;
- one load from Y ;
- one store to Z ;
- one floating-point addition.

If the dtype size is s bytes and there are N elements, the memory traffic is approximately:

$$\text{Bytes moved} = N(2s + s) = 3Ns.$$

For FP32, $s = 4$, so:

$$\text{Bytes moved} = 12N.$$

The number of floating-point operations is:

$$\text{FLOPs} = N.$$

The arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}} = \frac{N}{12N} = \frac{1}{12}.$$

This is very low, so vector addition is usually memory-bandwidth-bound.

Memory Coalescing

If consecutive lanes access consecutive memory addresses, memory accesses are coalesced.

For FP32 contiguous arrays:

$$\text{addr}(X_i) = \text{base}(X) + 4i.$$

For a warp accessing $i, i+1, \dots, i+31$, addresses are contiguous:

$$\text{base}(X) + 4i, \text{base}(X) + 4(i+1), \dots, \text{base}(X) + 4(i+31).$$

This is favorable for global memory throughput. A vector addition kernel written with contiguous offsets naturally exposes coalesced memory access.

Occupancy

Occupancy measures how many warps or CTAs can be resident on an SM. It is constrained by registers, shared memory, maximum resident warps, and maximum resident blocks.

A simplified occupancy bound is:

$$\text{resident warps} \leq \min \left(W_{\max}, \left\lfloor \frac{R_{\text{SM}}}{R_{\text{warp}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{S_{\text{CTA}}} \right\rfloor W_{\text{CTA}} \right).$$

Here:

- $W_{\max}$ is the architectural maximum resident warps per SM.
- R_{SM} is the register file capacity per SM.
- R_{warp} is register usage per warp.
- S_{SM} is shared memory capacity per SM.
- S_{CTA} is shared memory used by a CTA.
- W_{CTA} is the number of warps per CTA.

Triton compiler decisions can affect register pressure, shared memory usage, and therefore occupancy.

Mathematical Reasoning

Roofline Model

The roofline model estimates achievable performance as:

$$F_{\text{achievable}} = \min(F_{\text{max}}, IB_{\text{max}}).$$

Here:

- F_{max} is peak compute throughput.
- B_{max} is peak memory bandwidth.
- I is arithmetic intensity.

For vector addition with FP32:

$$I = \frac{1}{12}.$$

Therefore:

$$F_{\text{achievable}} = \min\left(F_{\text{max}}, \frac{B_{\text{max}}}{12}\right).$$

Since $B_{\text{max}}/12$ is generally much smaller than F_{max} , vector addition is memory-bound.

Common Subexpression Elimination

If a program computes the same expression twice:

$$a = b + c, \quad d = b + c,$$

the compiler can rewrite it as:

$$a = b + c, \quad d = a.$$

This transformation is called common subexpression elimination. It is valid when b and c have not changed between the two expressions.

Dead Code Elimination

If a value is computed but never used, it can be removed. Suppose:

$$e = 2a$$

and e has no effect on any output. Then the computation can be eliminated.

In graph terms, if $G = (V, E)$ is a data-dependency graph and a node $v \in V$ has no path to an output node, then v is removable if it has no side effects:

$v \not\rightsquigarrow y \implies v$ is removable.

Constant Propagation

If a value is known at compile time, operations involving it can be simplified. For example:

$$B = 1024, \quad \text{offsets} = pB + r.$$

Because B is known at compile time, later compiler passes can simplify and specialize the generated code.

Broadcast and Elementwise Reordering

The lecture discusses optimizations involving broadcasted values. If an elementwise operation is applied to broadcasted values, the compiler may perform the smaller operation first, then broadcast.

For example:

$$\text{broadcast}(a) + \text{broadcast}(b) = \text{broadcast}(a + b).$$

More generally, for a pure elementwise function f :

$$f(\text{broadcast}(x)) = \text{broadcast}(f(x)).$$

This reduces redundant computation.

Code Walkthrough

Representative Triton Vector Addition Kernel

```
import torch
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)

    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)

    mask = offsets < n_elements
```

```

x = tl.load(x_ptr + offsets, mask=mask)
y = tl.load(y_ptr + offsets, mask=mask)

out = x + y

tl.store(out_ptr + offsets, out, mask=mask)

```

Kernel Explanation

The decorator `@triton.jit` tells Triton that this function should be compiled rather than executed as normal Python.

The argument `BLOCK_SIZE: tl.constexpr` means that `BLOCK_SIZE` is a compile-time constant. Changing it can produce a new compiled kernel.

The line below gets the current program instance ID:

```
pid = tl.program_id(axis=0)
```

This behaves like a block-level identifier.

The following lines construct the vector of element indices handled by this program instance:

```

block_start = pid * BLOCK_SIZE
offsets = block_start + tl.arange(0, BLOCK_SIZE)

```

Mathematically, the offsets are:

$$[pB, pB + 1, \dots, pB + B - 1].$$

The mask guards memory operations so that the final block does not read or write past the end of the tensor:

```
mask = offsets < n_elements
```

The loads perform masked global memory reads:

```

x = tl.load(x_ptr + offsets, mask=mask)
y = tl.load(y_ptr + offsets, mask=mask)

```

The addition is an elementwise operation over the block:

```
out = x + y
```

The store writes the result back to global memory:

```
tl.store(out_ptr + offsets, out, mask=mask)
```

Wrapper and Launch Grid

```
def add(x, y):
    output = torch.empty_like(x)
    n_elements = output.numel()

    grid = lambda meta: (triton.cdiv(n_elements, meta["BLOCK_SIZE"]),)

    add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)

    return output
```

The launch grid contains:

$$G = \left\lceil \frac{N}{B} \right\rceil$$

program instances, where N is the number of elements and B is the block size.

The ceiling division operation is:

$$\text{cdiv}(N, B) = \left\lceil \frac{N}{B} \right\rceil = \left\lfloor \frac{N + B - 1}{B} \right\rfloor.$$

Compilation Artifacts

After compilation, a cache directory may contain files similar to:

```
add_kernel.ttir
add_kernel.ttgir
add_kernel.llir
add_kernel.ptx
add_kernel.cubin
add_kernel.json
```

The conceptual meaning is:

- `ttir`: common Triton IR.
- `ttgir`: Triton GPU IR.
- `llir`: LLVM IR.
- `ptx`: NVIDIA PTX assembly.
- `cubin`: compiled CUDA binary.
- `json`: metadata about target, backend, and artifacts.

Forcing Recompilation

A common debugging workflow is:

```
rm -rf ~/.triton/cache
python vector_add.py
find ~/.triton/cache -maxdepth 3 -type f
```

Clearing the cache is important because Triton may otherwise reuse an existing compiled artifact.

Dumping IR

To inspect IR after compiler passes, use:

```
MLIR_ENABLE_DUMP=1 python vector_add.py
```

The output can be large, but it shows the state of the program after many compiler passes. This helps answer questions such as:

- Did the compiler reorder loads or stores?
- Did canonicalization simplify the IR?
- Did common subexpression elimination remove redundancy?
- Did Triton GPU lowering introduce target-specific operations?
- Did the final lowering path reach LLVM and PTX as expected?

Inspecting Cubin, PTX, and SASS

Useful commands include:

```
cuobjdump --dump-ptx add_kernel.cubin
cuobjdump --dump-sass add_kernel.cubin
readelf -a add_kernel.cubin
```

These tools help inspect the low-level binary or assembly produced by the compiler.

Triton and MLIR Internals

Compiler Passes

Triton uses both standard MLIR passes and Triton-specific passes.

Standard compiler passes include:

- inlining;
- canonicalization;
- constant propagation;

- common subexpression elimination;
- dead code elimination;
- control-flow simplification.

Triton-specific passes include:

- combining pointer operations;
- simplifying broadcast and elementwise operations;
- loop unrolling;
- GPU pipelining;
- CTA planning;
- dot operand optimization;
- tensor-core-related lowering;
- target-specific GPU lowering.

A compiler pass can be modeled as:

$$\text{IR}_{\text{before}} \rightarrow \text{IR}_{\text{after}}.$$

TableGen

TableGen is a code-generation mechanism used in LLVM and MLIR projects. Triton uses TableGen-style definitions for some passes and dialect structures.

The typical pattern is:

TableGen declaration $\rightarrow$ generated boilerplate $\rightarrow$ C++ implementation $\rightarrow$ Python binding $\rightarrow$ compiler pipeline

This explains why a Triton compiler pass may appear in multiple parts of the codebase.

Python Bindings

Although many compiler internals are implemented in C++ and MLIR, Triton’s compiler orchestration is exposed through Python. Passes can be wrapped through bindings and called from the Python compiler pipeline.

This means that debugging Triton internals often requires reading:

- Python compiler pipeline code;
- MLIR TableGen files;

- C++ pass implementation files;
- backend driver code;
- runtime launcher code.

Backend-Specific Lowering

NVIDIA Backend

For NVIDIA GPUs, Triton lowers toward PTX and cubin. The backend may include NVIDIA-specific concepts such as compute capability, warp behavior, PTX generation, tensor core lowering, and CUDA driver loading.

A simplified NVIDIA path is:

Triton IR $\rightarrow$ Triton GPU IR $\rightarrow$ Triton NVIDIA GPU IR $\rightarrow$ LLVM IR $\rightarrow$ PTX $\rightarrow$ cubin.

Other Backends

Triton is increasingly modular. Other backends can lower from the common Triton representation into different target code. Conceptually:

$$\text{Triton IR} \rightarrow \left\{ \begin{array}{l} \text{CUDA-specific lowering,} \\ \text{AMD-specific lowering,} \\ \text{Intel GPU lowering,} \\ \text{CPU lowering,} \\ \text{custom accelerator lowering.} \end{array} \right.$$

This is why Triton is interesting as a portable kernel-generation layer.

Target Metadata

Compilation metadata may include:

- backend name;
- target architecture;
- compute capability;
- number of warps;
- threads per warp;
- generated artifact paths;
- kernel name;

- dot precision flags;
- shared library paths.

A JSON metadata file in the cache often records these details.

Performance Intuition

Why Simple Kernels Are Useful for Learning Internals

Vector addition is simple enough that every source-level operation maps clearly to IR:

program id $\rightarrow$ offsets $\rightarrow$ mask $\rightarrow$ loads $\rightarrow$ add $\rightarrow$ store.

This makes it ideal for learning how Triton represents and lowers programs.

When Triton Performance Comes From the Compiler

Triton performance is not only about the syntax. It comes from compiler decisions such as:

- avoiding unnecessary layout conversions;
- simplifying pointer arithmetic;
- removing redundant operations;
- improving memory access patterns;
- planning CTA layout;
- optimizing dot operands;
- using tensor cores when possible;
- pipelining memory operations and compute;
- specializing on compile-time constants.

Memory-Bound Versus Compute-Bound

A kernel is memory-bound when:

$$IB_{\max} < F_{\max}.$$

A kernel is compute-bound when:

$$IB_{\max} \geq F_{\max}.$$

Vector addition is memory-bound because its arithmetic intensity is low. Matrix multiplication can be compute-bound because it reuses loaded data many times.

For matrix multiplication:

$$C = AB,$$

with:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

The total FLOPs are approximately:

$$2MKN.$$

Because tiled GEMM can reuse loaded elements from A and B , its arithmetic intensity can be much higher than vector addition.

Tensor Cores

For operations such as `tl.dot`, Triton may lower high-level dot products into tensor-core-friendly instructions if shapes, dtypes, layouts, and target architecture allow it.

Tensor core usage depends on:

- dtype such as FP16, BF16, TF32, or INT8;
- tile shapes;
- operand layouts;
- memory alignment;
- target GPU architecture;
- dot precision settings;
- compiler lowering passes.

Common Misconceptions

Misconception: Triton Is Just Python CUDA

Triton is not simply Python syntax for CUDA. It is a compiler stack with frontend IR, MLIR-based transformations, backend lowering, runtime launchers, and a compilation cache.

Misconception: PTX Is Final Machine Code

PTX is not the final hardware instruction stream. It is further lowered or assembled into architecture-specific code.

Misconception: One Triton Function Means One Binary

One Triton function can generate multiple compiled artifacts depending on meta-parameters, constants, dtypes, and target hardware.

Misconception: MLIR Means Machine Learning IR

MLIR means Multi-Level Intermediate Representation. It is a general compiler framework, not a machine learning framework.

Misconception: The Cache Is Unimportant

The Triton cache strongly affects debugging. If a compiled kernel is already cached, Triton may not regenerate IR dumps or new artifacts unless the cache key changes or the cache is cleared.

Misconception: Compiler Passes Are Only Cleanup

Compiler passes can directly affect performance. They influence memory operations, layout conversions, tensor-core lowering, instruction count, register usage, and launch behavior.

Practical Takeaways

- Triton should be understood as a compiler stack, not only a kernel-writing language.
- The main lowering path is from Triton DSL to Triton IR, Triton GPU IR, LLVM IR, PTX, and cubin.
- MLIR provides the infrastructure that allows Triton to define custom dialects and compiler passes.
- Triton IR acts as a common layer before backend-specific lowering.
- Backend modularity is important for NVIDIA, AMD, Intel, CPU, and custom accelerator support.
- The Triton cache stores compiled artifacts and metadata.
- Clearing the cache is useful when debugging compilation behavior.
- The environment variable `MLIR_ENABLE_DUMP` is useful for inspecting the state of IR after passes.

- Vector addition is a good first kernel for mapping source code to IR.
- More complex kernels require studying layout transformations, dot lowering, CTA planning, and tensor-core use.

Project or Experiment Ideas

Experiment 1: Inspect Vector Addition

Compile a vector addition kernel and inspect every generated artifact. Try to map each source line to Triton IR and PTX.

Experiment 2: Change Block Size

Run the same kernel with different `BLOCK_SIZE` values. Observe whether the cache key and generated artifacts change.

Experiment 3: Change Dtype

Compare FP32, FP16, and BF16 versions of the same kernel. Inspect differences in IR, PTX, and metadata.

Experiment 4: Dump MLIR Passes

Run:

```
MLIR_ENABLE_DUMP=1 python vector_add.py
```

Then clear the cache and rerun. Observe which passes appear and how IR changes across stages.

Experiment 5: Inspect SASS

Use:

```
cuobjdump --dump-sass kernel.cubin
```

Map SASS instructions back to high-level operations such as loads, arithmetic, and stores.

Experiment 6: Implement a Diagnostic Pass

Modify a local Triton checkout to add a simple operation-counting pass. Insert it at different lowering stages and compare the operation counts.

Experiment 7: Compare Vector Add and Matmul

Inspect generated IR for vector addition and matrix multiplication. Compare:

- arithmetic intensity;
- number of memory operations;
- layout transformations;
- use of `tl.dot`;
- tensor-core-related lowering.

Final Mental Model

Triton starts as a Python-style kernel language, but its real power comes from its compiler pipeline. A user writes a block-level tensor program. Triton specializes it, lowers it through multiple IRs, applies common and target-specific compiler passes, emits backend-specific code, stores the result in a cache, and launches the resulting binary through the appropriate runtime.

The full mental model is:

Python-style Triton kernel $\rightarrow$ Triton frontend $\rightarrow$ common Triton IR $\rightarrow$ MLIR passes $\rightarrow$ GPU-aware IR $\rightarrow$ backend

For simple kernels, this pipeline turns offsets, masks, loads, arithmetic, and stores into executable instructions. For complex ML kernels, the same pipeline controls tiling, tensor core usage, layout transformations, memory movement, pipelining, register pressure, and occupancy.

The visible Triton code is the frontend. The compiler pipeline is the real engine. Understanding Triton internals means learning how each IR level gradually turns tensor-block intent into hardware-specific execution.

Visual Concept

Draw a left-to-right pipeline with boxes labeled Triton DSL, Triton IR, Triton GPU IR, NVIDIA GPU IR, LLVM IR, PTX, cubin, and GPU launch. Under the middle boxes, add smaller labels for canonicalization, common subexpression elimination, dead code elimination, layout conversion, dot optimization, CTA planning, and pipelining.